

# *LINGUAGENS LIVRES DO CONTEXTO*

## *GRAMÁTICAS*

---

Prof. Michael Móra  
michael.mora@pucrs.br

Material original desenvolvido pelos profs. Júlio Machado,  
Renata Vieira, Alexandre Agustini, Michael Móra e outros

# Linguagens Livres do Contexto

## **Estudo das Linguagens Livres do Contexto - Tipo 2:**

- universo mais amplo de linguagens do que as LR
- trata, adequadamente, questões típicas de linguagens de programação:
  - \* parênteses balanceados
  - \* construções bloco-estruturadas

## **Algoritmos reconhecedores e geradores**

- relativamente simples
- eficiência razoável

**Universo de Todas as Linguagens**

**Linguagens Livre do Contexto**

**Linguagens Regulares**

# Aplicações típicas

- centradas em linguagens artificiais
  - \* em especial, nas linguagens de programação
- analisadores sintáticos
- tradutores de linguagens
- processadores de texto em geral

## Hierarquia de Chomsky

Classe das Linguagens Livres do Contexto

contém propriamente a Classe das Linguagens Regulares

**Ainda é uma classe relativamente restrita**

- fácil definir linguagens que não pertencem a esta classe

# Abordagens

- Gramática Livre do Contexto (axiomático ou gerador)
  - \* regras de produção de palavras
- Autômato com Pilha (operacional ou reconhecedor)
  - \* análogo ao autômato finito não-determinístico
  - \* adicionalmente:  
memória auxiliar tipo pilha que pode ser lida ou gravada

# Gramática

- conjunto *finito* de regras
- quando aplicadas sucessivamente, geram palavras
- conjunto de todas as palavras geradas por uma gramática - define a linguagem

## **Gramáticas para linguagens naturais como Português**

- análogo as usadas para linguagens artificiais como Java

## **Gramáticas também são usadas para definir semântica**

- entretanto, em geral, são usados outros formalismos

# Gramática Livre do Contexto

$$G = (V, T, P, S)$$

- $V$ , conjunto *finito* de símbolos variáveis ou não-terminais
- $T$ , conjunto *finito* de símbolos terminais disjunto de  $V$
- $P: V \times (V \cup T)^*$ , relação *finita* de produções (ou regras de produções)
- $S$ , elemento distinguido de  $V$  variável inicial

Representação de uma regra de produção  $(\alpha, \beta)$  de  $P$

$$\alpha \rightarrow \beta$$

Representação abreviada para  $\alpha \rightarrow \beta_1, \alpha \rightarrow \beta_2, \dots, \alpha \rightarrow \beta_n$

$$\alpha \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

# Convenções

- $A, B, C, \dots, S, T$  para símbolos variáveis
- $a, b, c, \dots, s, t$  para símbolos terminais
- $u, v, w, x, y, z$  para palavras de símbolos terminais
- $\alpha, \beta, \dots$  para palavras de símbolos variáveis ou terminais



# Derivação

- aplicação de uma regra de produção é denominada **derivação**
- aplicação sucessiva de regras de produção
  - \* fecho transitivo da relação de derivação
  - \* permite derivar palavras da linguagem

# Relação de Derivação

$G = (V, T, P, S)$  gramática

Derivação é um par da **Relação de Derivação** denotada por  $\Rightarrow$

- domínio em  $(V \cup T)^+$  e codomínio em  $(V \cup T)^*$
- $(\alpha, \beta)$  é representado de forma infixada

$$\alpha \Rightarrow \beta$$

$\Rightarrow$  é indutivamente definida como segue:

- para toda produção da forma  $S \rightarrow \beta$  ( $S$  é o símbolo inicial de  $G$ )

$$S \Rightarrow \beta$$

- para todo par  $\eta \Rightarrow \rho \alpha \sigma$  da relação de derivação

\* se  $\alpha \rightarrow \beta$  é regra de  $P$ , então  $\eta \Rightarrow \rho \beta \sigma$

## Portanto, derivação

- substituição de uma subpalavra
- de acordo com uma regra de produção

## Sucessivos passos de derivação

- $\Rightarrow^*$  fecho transitivo e reflexivo da relação  $\Rightarrow$   
zero ou mais passos de derivações sucessivos
- $\Rightarrow^+$  fecho transitivo da relação  $\Rightarrow$   
um ou mais passos de derivações sucessivos
- $\Rightarrow^i$   
exatos  $i$  passos de derivações sucessivos ( $i$  um número Natural)

# Linguagem Gerada

$G = (V, T, P, S)$  gramática

Linguagem Gerada por  $G$ :  $L(G)$  ou  $GERA(G)$

- palavras de símbolos terminais deriváveis a partir de  $S$

$$L(G) = \{ w \in T^* \mid S \Rightarrow^+ w \}$$

# Exemplo 1

## Gramática, Derivação, Linguagem Gerada:

$$G = (V, T, P, N)$$

- $V = \{ N, D \}$
- $T = \{ 0, 1, 2, \dots, 9 \}$
- $P = \{ N \rightarrow D, N \rightarrow DN, D \rightarrow 0 \mid 1 \mid \dots \mid 9 \}$

Gera, sintaticamente, o conjunto dos números naturais

- se distinguem os zeros à esquerda
- 123 diferente 0123

$$G = (V, T, P, N)$$

- $V = \{ N, D \}$
- $T = \{ 0, 1, 2, \dots, 9 \}$
- $P = \{ N \rightarrow D, N \rightarrow DN, D \rightarrow 0 \mid 1 \mid \dots \mid 9 \}$

Uma derivação do número 243

- $N \Rightarrow N \rightarrow DN$
- $DN \Rightarrow D \rightarrow 2$
- $2N \Rightarrow N \rightarrow DN$
- $2DN \Rightarrow D \rightarrow 4$
- $24N \Rightarrow N \rightarrow D$
- $24D \Rightarrow D \rightarrow 3$
- 243

Portanto

- $S \Rightarrow^* 243$
- $S \Rightarrow^+ 243$
- $S \Rightarrow^6 243$

Interpretação indutiva da gramática

- *Base de Indução*: todo dígito é natural
- *Passo de Indução*: se  $n$  é natural, então a concatenação com qualquer dígito também é natural

## "Livre do contexto"

- mais geral classe de linguagens cuja produção é da forma  $A \rightarrow \alpha$
- em uma derivação, a variável  $A$  deriva  $\alpha$ 
  - \* sem depender ("livre") de qualquer análise dos símbolos que antecedem ou sucedem  $A$  (o "contexto") na palavra que está sendo derivada



# Exemplo 2

## Duplo Balanceamento

$$L = \{ a^n b^n \mid n \geq 0 \}$$

$$G = (\{ S \}, \{ a, b \}, P, S)$$

- $P = \{ S \rightarrow aSb \mid S \rightarrow \varepsilon \}$
- $L(G) = L$

Derivação da palavra aabb

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aa\varepsilon bb = aabb$$

## Importante: Duplo Balanceamento

- analogia com estruturas de duplo balanceamento
  - \* em linguagens de programação
- linguagens bloco-estruturadas
  - \* **begin<sup>n</sup> end<sup>n</sup> e similares**
- linguagens com parênteses balanceados
  - \* **(<sup>n</sup>)<sup>n</sup>**

# Gramáticas Equivalentes

G1 e G2 são Gramáticas Equivalentes se e somente se  
 $L(G1) = L(G2)$

# Exemplo 3

## Expressões Aritméticas

L = expressões aritméticas com colchetes balanceados, dois operadores e um operando

$$G = (\{ E \}, \{ +, *, [, ], x \}, P, E)$$

$$\bullet P = \{ E \rightarrow E+E \mid E * E \mid [E] \mid x \}$$

Derivação da expressão  $[x+x]*x$

$$E \Rightarrow E * E \Rightarrow [E] * E \Rightarrow [E+E] * E \Rightarrow [x+E] * E \Rightarrow [x+x] * E \Rightarrow [x+x] * x$$

- existe outra seqüência de derivação? Quantas?
- quais produções controlam o duplo balanceamento de colchetes?

# Relativamente às GLC

- Árvore de derivação
  - \* representa a derivação de uma palavra na forma de **árvore**
  - \* parte do símbolo inicial como a raiz
  - \* termina em símbolos terminais como folhas
- Gramática Ambígua
  - \* pelo menos uma palavra com duas ou mais árvores de derivação

# Árvore de Derivação

## **Derivação de palavras na forma de árvore**

- partindo do símbolo inicial como a raiz
- terminando em símbolos terminais como folhas

## **Conveniente em muitas aplicações**

- Compiladores
- processadores de textos

# Árvore de Derivação

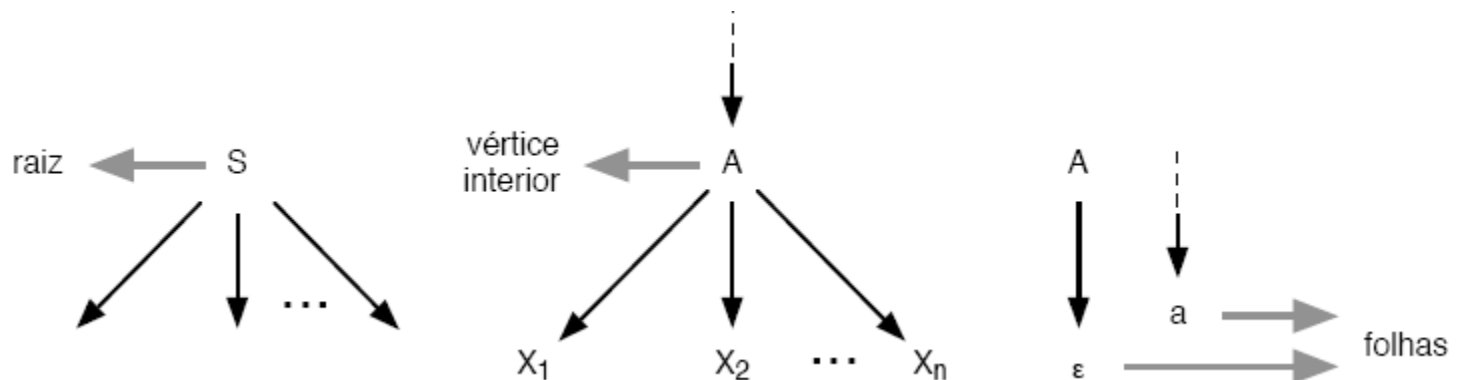
Raiz: símbolo inicial

Vértices interiores: variáveis

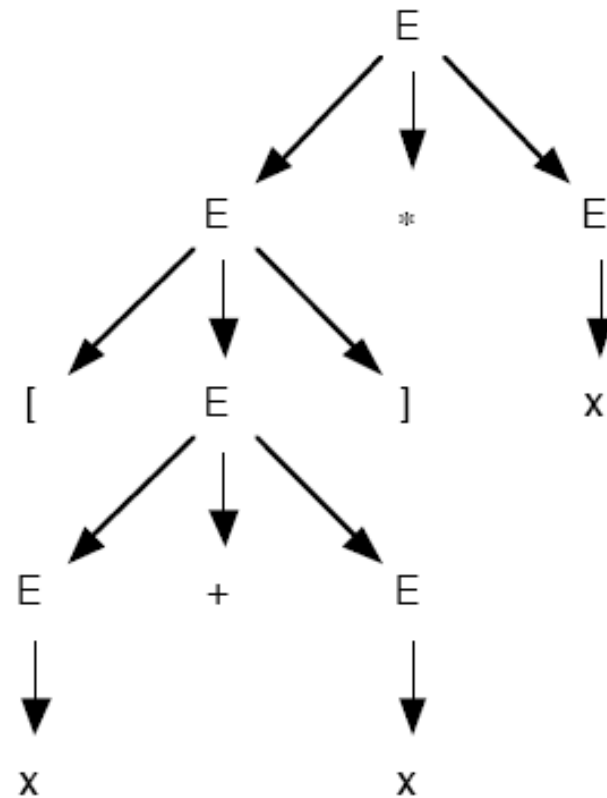
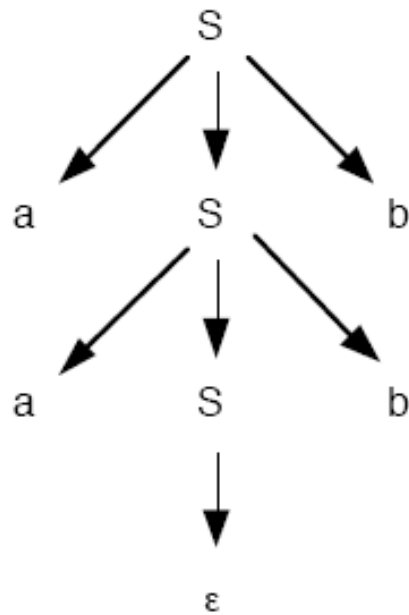
- se  $A$  é um vértice interior e  $X_1, X_2, \dots, X_n$  são os "filhos" de  $A$ 
  - \*  $A \rightarrow X_1X_2\dots X_n$  é uma produção da gramática
  - \*  $X_1, X_2, \dots, X_n$  são ordenados da esquerda para a direita

Vértice folha ou folha: terminal ou o símbolo vazio

- se vazio: único filho de seu pai ( $A \rightarrow \varepsilon$ )



# Exemplo Árvore de Derivação: aabb e $[x+x]*x$





## **Uma árvore de derivação**

- pode representar derivações distintas de uma mesma palavra

# Exemplo

## Árvore de Derivação × Derivações: $x+x*x$

$E \Rightarrow E+E \Rightarrow x+E \Rightarrow x+E*E \Rightarrow x+x*E \Rightarrow x+x*x$

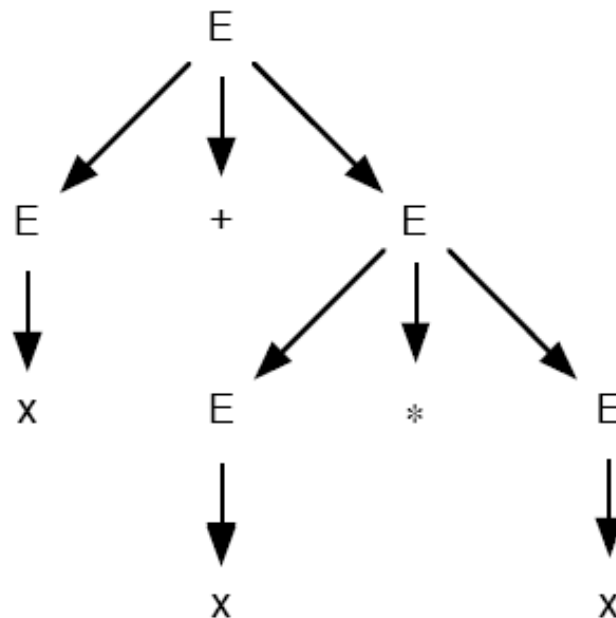
mais a esquerda

$E \Rightarrow E+E \Rightarrow E+E*E \Rightarrow E+E*x \Rightarrow E+x*x \Rightarrow x+x*x$

mais a direita

$E \Rightarrow E+E \Rightarrow E+E*E \Rightarrow x+E*E \Rightarrow x+x*E \Rightarrow x+x*x$

etc...



# Gramática Ambígua

## Gramática ambígua

- uma palavra associada a duas ou mais árvores de derivação

## **Pode ser desejável que a gramática usada seja não-ambígua**

- desenvolvimento e otimização de alguns algoritmos de reconhecimento

## **Nem sempre é possível eliminar ambigüidades**

- é fácil definir linguagens para as quais qualquer GLC é ambígua

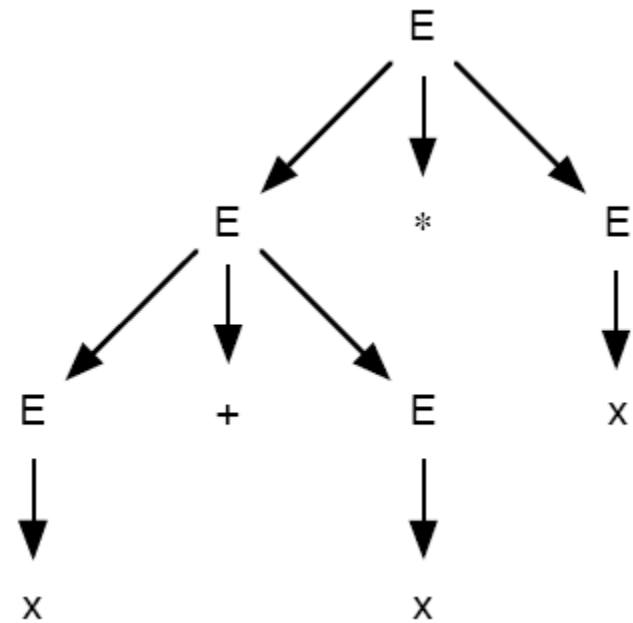
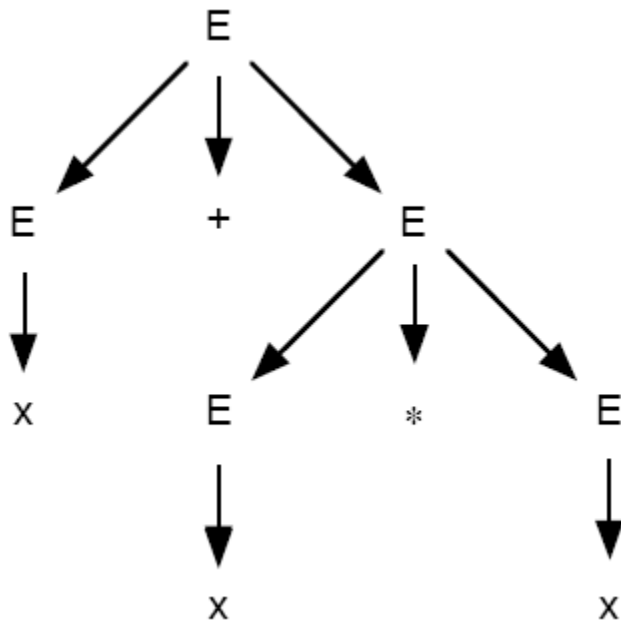
# GLC Ambígua

## Gramática (Livre do Contexto) Ambígua

Existe pelo menos uma palavra que possui duas ou mais árvores de derivação

**Exemplo Gramática Ambígua:  $x+x*x$**

# GLC Ambígua



# GLC Ambígua

## Mais de uma derivação à esquerda (direita)

$x+x*x$

- Derivação mais à esquerda

- \*  $E \Rightarrow E+E \Rightarrow x+E \Rightarrow x+E*E \Rightarrow x+x*E \Rightarrow x+x*x$

- \*  $E \Rightarrow E*E \Rightarrow E+E*E \Rightarrow x+E*E \Rightarrow x+x*E \Rightarrow x+x*x$

- Derivação mais à direita

- \*  $E \Rightarrow E+E \Rightarrow E+E*E \Rightarrow E+E*x \Rightarrow E+x*x \Rightarrow x+x*x$

- \*  $E \Rightarrow E*E \Rightarrow E*x \Rightarrow E+E*x \Rightarrow E+x*x \Rightarrow x+x*x$

# GLC Ambígua

## **Forma equivalente de definir gramática ambígua**

- existe pelo menos uma palavra com duas ou mais derivações mais à esquerda
  - \* alternativamente, mais à direita

## **Teorema: Gramática Ambígua**

Uma GLC é uma Gramática Ambígua se existe pelo menos uma palavra com

- duas ou mais derivações mais à esquerda ou
- duas ou mais derivações mais à direita

# GLC Ambígua

## Linguagem Inerentemente Ambígua

Qualquer GLC é ambígua

## Ex: Linguagem Inerentemente Ambígua

$\{ w \mid w = a^n b^n c^m d^m \text{ ou } w = a^n b^m c^m d^n, n \geq 1, m \geq 1 \}$



# GLC Ambígua

**Ex: Linguagem Inerentemente Ambígua: contra-exemplo**

Expressões aritméticas é não-ambígua (exercício)

Correto entendimento da solução é especialmente importante!

- justifica a maneira aparentemente “estranha” de definir expressões na maioria das gramáticas das linguagens de programação